

■ 강의명: CSCI E-89: 인공지능 개론

■ 주차: Lecture 1

■ 교수명: UIUC Student

■ 목적: 본 문서는 CSCI E-89 '인공지능 개론' 1강의 핵심 내용을 통합 정리한 LaTeX 노트입니다. 강의 운영 방식, 평가 기준, 필수 선수 지식부터 신경망의 기본 원리, 활성화 함수, 손실 함수, 최적화 알고리즘까지 다룹니다. 특히, 비전공자도 쉽게 이해할 수 있도록 개념 설명에 중점을 두었으며, 실제 과제 수행에 필요한 코드 예시와 주의사항을 포함하여 시험 대비 및 실습에 즉시 활용 가능하도록 구성했습니다. 모든 내용은 강의 자료를 기반으로 하며, 추가 설명과 비유를 통해 이해도를 높였습니다.

Contents

1 용어 정리	2
---------------	---

1 용어 정리

Table 1: 주요 용어 및 설명

용어	쉬운 설명	원어	비고
신경망	사람 뇌의 뉴런 연결 방식을 모방한 컴퓨터 모델	Neural Network	복잡한 패턴 학습에 사용
활성화 함수	신경망 뉴런의 출력을 결정하는 비선형 함수	Activation Function	비선형성 도입으로 복잡한 문제 해결
손실 함수	모델의 예측이 실제 값과 얼마나 다른지 측정하는 함수	Loss Function	개별 데이터 포인트의 오차
비용 함수	전체 데이터셋에 대한 손실 함수의 평균	Cost Function	모델 학습의 최종 목표
경사 하강법	비용 함수를 최소화하기 위해 기울기를 따라 내려가는 최적화 방법	Gradient Descent	모델 파라미터 업데이트 방식
확률적 경사 하강법	한 번에 하나의 데이터 포인트로 경사를 계산하여 업데이트	Stochastic Gradient Descent (SGD)	계산 효율적, 지역 최저점 탈출 용이
미니 배치 경사 하강법	작은 데이터 묶음(미니 배치)으로 경사를 계산하여 업데이트	Mini-Batch Gradient Descent	GD와 SGD의 장점 결합
하이퍼파라미터	모델 학습 전에 사용자가 직접 설정해야 하는 값	Hyperparameter	학습률, 배치 크기 등
학습률	경사 하강법에서 한 번에 이동하는 스텝의 크기	Learning Rate	너무 크면 발산, 너무 작으면 느림
원-핫 인코딩	범주형 데이터를 이진 벡터로 표현하는 방식	One-Hot Encoding	분류 문제의 출력 레이블 표현
소프트맥스	여러 클래스에 대한 확률 분포를 출력하는 활성화 함수	Softmax Function	분류 문제의 최종 출력층에 사용
교차 엔트로피	분류 문제에서 주로 사용되는 손실 함수	Cross-Entropy Loss	예측 확률과 실제 레이블 간의 차이 측정
구조적 토픽 모델링	텍스트에서 주제를 자동으로 추출하는 통계 모델	Structural Topic Modeling (STM)	R 언어를 사용하여 구현

강의 개요 및 운영

강의 목표 및 내용

이 강의는 신경망(Neural Networks)에 대한 소개를 목표로 합니다. 수강생들이 신경망에 대한 사전 지식이 없다고 가정하고, 기본적인 개념부터 심층적으로 다룰 예정입니다. 특히 자연어 처리(Natural Language Processing, NLP) 분야에 신경망이 어떻게 적용되는지에 초점을 맞춥니다.

강의 자료 및 접근

강의 자료는 Canvas를 통해 제공되며, 다음을 포함합니다:

- **강의 슬라이드:** 강의에서 사용되는 모든 슬라이드.
- **예시 코드:** 강의 내용과 관련된 코드 예시.
- **조교(TA) 자료:** TA 세션에서 사용되는 추가 파일.

모든 자료는 강의 후 Canvas의 'Files' 탭에서 다운로드할 수 있습니다. 제공된 코드는 자유롭게 활용할 수 있으며, 필요시 출처를 명시하면 됩니다.

□ 핵심 정보

코드 활용 팁: 강의에서 제공된 코드를 수정하여 과제를 제출하는 것은 허용됩니다. 이는 정책 위반이 아니며, 수업에서 제시된 예시나 TA가 제공한 자료를 기반으로 작업하는 것을 권장합니다.

소통 채널

강의 관련 소통은 다음 채널을 통해 이루어집니다:

- **공지 (Announcements):** Canvas의 'Announcements' 탭을 통해 중요한 정보가 게시됩니다. 이메일로도 발송되므로 주기적으로 확인하거나 이메일을 주의 깊게 살펴보세요.
- **WhatsApp 그룹:** 수강생 간의 자유로운 소통을 위해 개설됩니다. 교수진이 상시 모니터링하지는 않지만, 필요시 참고 자료를 공유하거나 답변을 제공할 수 있습니다.
- **Piazza:** 공식적인 질문 및 답변 포럼입니다. 교수진과 TA가 질문에 답변할 의무를 느끼는 채널이므로, 강의 내용에 대한 질문은 Piazza를 통해 하는 것이 가장 좋습니다. 다른 학생들도 답변을 볼 수 있어 효율적입니다.
- **이메일:** 교수진이나 TA에게 직접 문의할 때 사용합니다.
- **Canvas Inbox:** Canvas 내에서 교수진이나 TA에게 직접 메시지를 보낼 수 있습니다.

주의사항

학생 간 연락: Canvas에서는 학생 이름이 서로에게 숨겨져 있어 직접 연락할 수 없습니다. 다른 학생과 소통하고 싶다면 WhatsApp 그룹을 활용하세요. 연락처 공유는 자율에 맡겨집니다.

조교(TA) 및 교수 세션

- **조교 (TA):** 총 5명의 TA가 있으며, Andrea가 수석 TA(Head TA)입니다. Andrea는 딥러닝 및 자연어 처리 분야에서 5년 이상 TA 경험이 있는 베테랑이므로, 행정적 질문이나 누구에게 물어야 할지 모르는

질문은 Andrea에게 먼저 이메일로 문의하세요.

- **TA 세션:** 주 2회 진행됩니다 (보통 수/목요일, 토/일요일). 강의 내용을 보충하고 구현 예시를 다룹니다. TA는 강의 계획을 따르지만, 세션 내용은 TA의 재량에 따라 유연하게 진행됩니다. TA들은 박사 학위 소지자도 있을 정도로 전문성이 높습니다.
- **교수 세션:** 매주 금요일 1시간 동안 진행됩니다. 주로 파이썬(Python) 구현 예시를 시연하고 토론하며, 이론은 주로 화요일 강의에서 다룹니다.

□ 핵심 정보

세션 활용: 모든 세션은 녹화되어 제공되므로, 실시간 참석이 어렵더라도 나중에 시청할 수 있습니다. 질문이 있다면 실시간 세션에 참여하는 것이 가장 좋습니다.

수업 참여 방식

이 강의는 온라인 옵션을 제공하므로, 모든 세션(강의, TA 세션, 교수 세션)에 직접 참석할 필요가 없습니다. 모든 세션은 녹화되어 Canvas의 'Class Recordings' 링크를 통해 다시 볼 수 있습니다. 질문이 있을 경우, 다른 학생의 발언을 존중하며 문장 중간에 끊지 않는 선에서 언제든지 질문할 수 있습니다.

평가 및 마감

성적 배점

강의 성적은 다음 요소들로 구성됩니다:

- 과제 (Homework): 65%
- 퀴즈 (Quizzes): 20%
- 최종 프로젝트 (Final Project): 15%

과제 (Assignments)

- 마감: 매주 일요일 (보스턴 시간 기준)에 마감됩니다. 첫 과제는 9월 14일입니다.
- 제출:
 - 보고서: Microsoft Word 또는 PDF 형식으로 제출합니다. 코드의 핵심 부분, 결과(플롯, 표), 간략한 설명을 포함해야 합니다. 주피터 노트북(Jupyter Notebook)을 PDF로 변환하여 제출하는 것도 허용됩니다.
 - 원본 코드: 주피터 노트북 파일(.ipynb) 또는 여러 개의 파이썬(.py) 파일인 경우 압축 파일(.zip) 형태로 제출해야 합니다. TA가 코드를 직접 실행하여 검토할 수 있도록 합니다.
- 지연 페널티: 과제 제출이 늦어지면 페널티가 부과됩니다 (예: 3일 지연 시 30% 감점).
- 모범 답안 공유: 매주 5개 정도의 모범 답안이 Canvas에 공유됩니다. 이는 신경망 최적화의 비볼록성(non-convexity)으로 인해 학생마다 다른 결과가 나올 수 있기 때문에, 서로의 접근 방식을 학습하고 비교하는 데 도움이 됩니다. 자신의 답안 공유를 원치 않으면 TA에게 문의하세요.
- 성능 목표: 일부 과제에서는 특정 성능(예: 85% 정확도)을 달성하도록 요구할 수 있습니다. 이는 가장 어려운 부분이 될 수 있으며, 다양한 아키텍처나 파라미터 초기화 방법을 실험해야 할 수도 있습니다.

□ 예제

과제 제출 예시:

- 보고서: 문제 해결 과정, 주요 코드 스냅샷, 결과 플롯, 표, 그리고 결과에 대한 간략한 논의를 포함합니다.
- 코드: 보고서에 포함된 코드의 원본 파일(예: Jupyter Notebook)을 제출합니다. 여러 파일인 경우 하나의 ZIP 파일로 묶습니다.

퀴즈 (Quizzes)

- 마감: 다음 강의(화요일) 전까지 마감됩니다. 첫 퀴즈는 9월 9일 20시 10분(보스턴 시간)까지입니다.
- 엄격한 마감: 퀴즈 마감은 매우 엄격합니다. 마감 후에는 재응시나 대체 퀴즈가 제공되지 않습니다. 질문이 다음 강의에서 논의되기 때문입니다.
- 응시 조건: 퀴즈는 7일 동안 언제든지 열어 풀 수 있으며, 시간 제한이나 감독이 없습니다. 15분 정도의 시간을 할애하여 마감 기한 내에 제출해야 합니다.

주의사항

퀴즈 마감 엄수: 퀴즈는 마감 기한을 놓치면 재응시 기회가 없습니다. 이는 강의 정책이며, 질문이 공개적으로 논의되기 때문에 대체 퀴즈를 만드는 것은 공정성 문제와 막대한 작업량을 야기합니다.

최종 프로젝트 (Final Project)

- **마감:** 강의 마지막 날인 12월 16일(월요일)에 마감됩니다. 이는 매우 엄격한 마감 기한입니다.
- **주제 선정:** 11월 25일(추수감사절 직전)까지 프로젝트 주제를 선정해야 합니다. 11월 9일에 제안된 주제 목록이 게시될 예정이며, 자신만의 주제를 제안하는 것도 적극 권장됩니다.
- **내용:** 자연어 처리(NLP)와 명시적으로 관련된 프로젝트여야 합니다. 신경망 외의 다른 언어 처리 기법(예: 구조적 토픽 모델링)도 허용됩니다.
- **제출물:**
 - 보고서: 7~15페이지 분량.
 - 슬라이드: 발표에 사용될 슬라이드.
 - 영상 발표: 10~15분 분량의 발표 영상.
 - 작동하는 데모 코드: 실행 가능한 코드.

주의사항

최종 프로젝트 마감: 최종 프로젝트 마감은 학교 정책상 매우 엄격합니다. 마감 기한을 놓칠 경우, 학교의 승인을 받아야만 연장이 가능합니다.

AI 기술 사용 정책

- **사용 권장:** 이 강의에서는 AI 도구(예: 생성형 AI) 사용을 적극 권장합니다. 이는 직장에서 이러한 도구를 사용하는 것이 불가피해질 것이기 때문입니다.
- **인용 필수:** AI 도구를 사용하여 생성된 코드, 텍스트 등을 제출할 때는 반드시 사용한 AI 도구를 명시하고 인용해야 합니다.
- **최종 책임:** AI가 생성한 결과물에 대한 최종 책임은 학생 본인에게 있습니다. "AI가 그랬다"고 변명할 수 없습니다.
- **기술 학습:** AI 도구를 효과적으로 사용하는 것은 중요한 기술입니다. 원하는 바를 명확하게 설명하고, 예시를 제공하며, 반복적인 수정을 통해 AI의 도움을 받는 방법을 배우는 것이 중요합니다.

선수 지식 및 교재

필수 선수 지식

- **미적분학 (Calculus):** 신경망의 역전파(backpropagation) 원리를 이해하는 데 도움이 됩니다. 이 강의에서는 역전파를 직접 구현하도록 요구하지 않으므로, 미적분학 지식이 '매우' 중요하지는 않습니다. 하지만 책에 제시된 공식을 이해하고 기본적인 용어(예: 미분)를 아는 수준이면 충분합니다.
- **확률론 (Probability Theory):** 생성 모델(generative models)을 다룰 때 필요합니다. 무작위 변수(random variable), 정규 분포(normal distribution)와 같은 기본 개념에 대한 직관적인 이해가 필요합니다. 엄밀한 정의를 알 필요는 없습니다.
- **파이썬 (Python):** 가장 중요한 선수 지식입니다. 이 강의에서는 파이썬 기초를 다루지 않으므로, 코딩 능력이 있다고 가정합니다. CSCI E-7과 같은 파이썬 입문 과정을 수강한 수준을 기대합니다.

권장 교재

이 강의에서는 특정 교재를 필수로 지정하지 않습니다. 하나의 교재가 모든 내용을 특정 순서나 방식으로 다루기 어렵기 때문입니다. 다음은 인기 있는 교재 목록이며, 수강생들은 다양한 자료를 활용하여 주제별로 학습하는 것을 권장합니다.

- **Bird 저서:** 매우 인기 있는 책입니다.
- **Transformers를 이용한 자연어 처리 (Natural Exposing with Transformers):** 트랜스포머(Transformers) 모델을 간략하게 다룰 예정입니다.
- **Text as Data:** 사회 과학자를 위해 쓰인 책이지만, 구조적 토픽 모델링(Structural Topic Modeling, STM)에 대한 중요한 챕터(13장)가 포함되어 있습니다. 이 책을 구매할 필요는 없으며, 해당 주제에 대한 원본 연구 논문을 통해 동일한 정보를 얻을 수 있습니다.
- **선택 교재:** 수학적 내용이 더 많아 읽기 어려울 수 있는 책입니다. 선택 사항입니다.

□ 핵심 정보

학습 전략: 특정 책의 챕터를 따라가기보다는, 강의에서 다루는 '주제'를 중심으로 학습하는 것이 효과적입니다. 예를 들어, 순환 신경망(Recurrent Neural Network)을 배운다면, 여러 책이나 온라인 자료, 연구 논문 등 다양한 출처를 통해 해당 주제를 깊이 이해하는 데 집중하세요. 이는 다양한 관점과 설명 방식을 접하며 자신에게 맞는 학습 방법을 찾는 데 도움이 됩니다.

R 언어 사용

이 강의에서 R 언어는 구조적 토픽 모델링(STM)을 다룰 때만 사용됩니다. STM은 R에 구현된 패키지를 사용하므로, 이 특정 모델을 다룰 때만 R을 사용하게 됩니다. R에 대한 사전 지식이 없어도 괜찮으며, 필요한 부분만 간략하게 소개될 예정입니다. 만약 최종 프로젝트 주제로 STM을 선택한다면 R을 사용해야 합니다.

신경망(Neural Networks) 소개

수동 특징 설계의 한계

선형 회귀 예시

신경망의 필요성을 이해하기 위해, 먼저 선형 회귀(Linear Regression) 모델을 살펴봅시다. 우리가 예측하려는 출력($\hat{y}$)이 입력(x)의 다항식 형태로 주어지는 경우를 가정해봅시다:

$$\hat{y} = w_0 + w_1x + w_2x^2 + w_3x^3$$

여기서 x 는 입력 데이터이고, w_0, w_1, w_2, w_3 는 모델이 학습해야 할 파라미터(가중치)입니다. 이 모델은 x, x^2, x^3 을 각각 특징(feature) u_1, u_2, u_3 으로 간주하여 선형 모델을 피팅하는 것과 같습니다. 즉, $u_1 = x, u_2 = x^2, u_3 = x^3$ 으로 정의하면, $\hat{y} = w_0 + w_1u_1 + w_2u_2 + w_3u_3$ 가 됩니다. 이러한 모델은 파라미터 w 에 대해 선형이므로 '선형 회귀'라고 불립니다.

□ 예제

수동 특징 설계의 문제점:

- **수동 설계:** x^2, x^3 과 같은 특징을 사람이 직접 설계해야 합니다. 이는 데이터를 시각적으로 분석하여 적절한 형태를 추측해야 함을 의미합니다.
- **고차원 데이터의 어려움:** 이미지(수많은 픽셀), 언어(단어 벡터 시퀀스)와 같은 고차원 데이터에서는 시각적 분석이나 수동 특징 설계가 거의 불가능합니다.
- **비선형 파라미터의 복잡성:** 만약 x 의 거듭제곱($x^{p_1}, x^{p_2}, x^{p_3}$)에서 p_1, p_2, p_3 까지 파라미터로 학습해야 한다면, 문제는 매우 비선형적이고 복잡해져 수치적으로 해결하기 어려워집니다.

신경망의 기본 아이디어: 자동 특징 학습

신경망은 이러한 수동 특징 설계의 한계를 극복하기 위해 고안되었습니다. 핵심 아이디어는 모델이 특징을 자동으로 학습하도록 하는 것입니다. 이를 위해 신경망은 '비선형성'을 도입하면서도 '중간 단계'에서는 선형 변환'을 사용하는 방식을 채택합니다.

- **비선형성 도입:** 신경망은 입력에 비선형 함수(f)를 적용하여 새로운 특징을 생성합니다.
- **중간 선형 변환:** 이 비선형 함수는 입력의 '선형 결합'에 적용됩니다.

$$u_1 = f(w_{10} + w_{11}x_1 + w_{12}x_2 + \dots)$$

$$u_2 = f(w_{20} + w_{21}x_1 + w_{22}x_2 + \dots)$$

그리고 최종 출력 $\hat{y}$ 또한 이러한 중간 특징($u_1, u_2, \dots$)의 선형 결합에 비선형 함수를 적용하여 얻습니다.

$$\hat{y} = f(w_0 + w_1u_1 + w_2u_2 + \dots)$$

여기서 w 값들은 모델이 학습해야 할 파라미터입니다.

□ 핵심 정보

왜 중간에 선형 변환을 사용할까? 이러한 구조는 미분(differentiation)을 쉽게 하기 위함입니다. 미적분학의 연쇄 법칙(chain rule)을 적용하면, 비선형 함수 f 가 선형 결합에 적용된 형태는 쉽게 미분할 수 있습니다. 이는 신경망의 파라미터를 효율적으로 최적화하는 데 결정적인 역할을 합니다.

신경망의 구조 (시각적 표현)

신경망은 여러 층(layer)으로 구성되며, 각 층은 여러 뉴런(neuron)을 포함합니다.

- **입력층 (Input Layer):** 외부 데이터를 받아들이는 부분입니다 (예: x_1, x_2).
- **은닉층 (Hidden Layer):** 입력층과 출력층 사이에 위치하며, 자동으로 특징을 학습하는 역할을 합니다 (예: $u_1, u_2, \dots$).
- **출력층 (Output Layer):** 최종 예측 값을 생성합니다 (예: $\hat{y}$).

각 뉴런은 이전 층의 뉴런들로부터 입력을 받아 선형 결합을 수행한 후, 활성화 함수(f)를 적용하여 다음 층으로 출력을 전달합니다. 각 연결선은 가중치(weight, w)를 가지며, 각 뉴런은 편향(bias, w_0)을 가집니다.

Figure 1: 신경망의 시각적 구조 (예시)

(이미지 설명: 입력 x_1, x_2 가 여러 은닉 뉴런 $u_1, u_2, \dots$ 으로 연결되고, 이 은닉 뉴런들이 다시 출력 $\hat{y}$ 로 연결되는 다층 구조를 보여주는 다이어그램. 각 연결선에는 가중치가, 각 뉴런에는 활성화 함수가 적용됨.)

신경망은 본질적으로 '함수의 중첩(nested functions)' 형태를 가집니다. 즉, 함수가 선형 결합에 적용되고, 그 결과가 다시 다른 함수의 선형 결합에 적용되는 방식입니다. 충분히 많은 뉴런과 층을 사용하면, 신경망은 원하는 어떤 비선형 함수라도 원하는 정확도로 근사할 수 있다는 '범용 근사 정리(Universal Approximation Theorem)'가 존재합니다.

활성화 함수 (Activation Functions)

개념 및 역할

활성화 함수(f)는 신경망의 각 뉴런에서 선형 결합된 입력 값에 적용되는 비선형 함수입니다.

- **비선형성 도입:** 활성화 함수가 없다면, 아무리 층을 깊게 쌓아도 결국 선형 모델과 동일한 결과를 내게 됩니다. 활성화 함수는 신경망이 복잡한 비선형 관계를 학습할 수 있도록 해줍니다.
- **생물학적 뉴런의 비유:** 역사적으로 활성화 함수는 생물학적 뉴런의 '활성화' 개념에서 유래했습니다. 뉴런은 특정 임계값 이상의 전기적 신호가 들어오면 활성화되어 다음 뉴런으로 신호를 전달합니다.
- **편향 (Bias):** 편향(w_0)은 뉴런의 활성화 임계값을 조절하는 역할을 합니다. 이는 선형 결합의 결과가 0이 아닌 다른 지점에서 활성화되도록 기준점을 이동시킵니다.

활성화 함수의 종류 및 특징

다양한 활성화 함수가 있으며, 각각의 특성에 따라 신경망의 학습 방식과 성능에 영향을 미칩니다.

1. 헤비사이드(Heaviside) 함수 (계단 함수)

- 정의: 입력 Z 가 0보다 작으면 0, 0보다 크거나 같으면 1을 출력합니다.

$$f(Z) = \begin{cases} 0 & \text{if } Z < 0 \\ 1 & \text{if } Z \geq 0 \end{cases}$$

- 문제점: 이 함수는 0에서 미분 불가능하며, 다른 모든 지점에서는 미분값이 0입니다. 이는 경사 하강법 (Gradient Descent)과 같은 최적화 알고리즘이 기울기를 통해 학습할 수 없게 만듭니다 (기울기가 0이므로 파라미터가 업데이트되지 않음). 따라서 실제 신경망에서는 사용되지 않습니다.

2. 렐루(ReLU: Rectified Linear Unit)

- 정의: 입력 Z 가 0보다 작으면 0, 0보다 크거나 같으면 Z 를 출력합니다.

$$f(Z) = \max(0, Z)$$

- 특징: 계산이 매우 효율적이고, 양수 영역에서는 기울기가 1로 일정하여 기울기 소실 (vanishing gradient) 문제를 완화합니다.
- 문제점: 음수 영역에서는 기울기가 0이므로, 한 번 음수 값이 들어오면 해당 뉴런이 더 이상 활성화되지 않는 '죽은 ReLU(Dying ReLU)' 문제가 발생할 수 있습니다. 하지만 확률적 최적화 기법 덕분에 실제로는 잘 작동합니다.

3. 리키 렐루(Leaky ReLU)

- 정의: ReLU의 '죽은 ReLU' 문제를 해결하기 위해, 음수 영역에서도 작은 기울기(α)를 가집니다.

$$f(Z) = \begin{cases} \alpha Z & \text{if } Z < 0 \\ Z & \text{if } Z \geq 0 \end{cases}$$

여기서 α 는 0과 1 사이의 작은 양수 값(예: 0.01)입니다.

- 특징: 음수 영역에서도 기울기가 존재하여 뉴런이 죽는 것을 방지합니다.
- 하이퍼파라미터: α 는 사용자가 직접 설정해야 하는 하이퍼파라미터입니다.

4. 맥스아웃(Maxout)

- 정의: 여러 선형 함수의 최댓값을 출력합니다.

$$f(Z_1, Z_2, \dots, Z_k) = \max(Z_1, Z_2, \dots, Z_k)$$

여기서 Z_i 는 각각 다른 가중치와 편향을 가진 선형 결합입니다.

- 특징: ReLU와 Leaky ReLU를 일반화한 형태로, 더 많은 유연성을 제공하지만 파라미터 수가 증가합니다.

5. 시그모이드(Sigmoid)

- 정의: 입력 Z 를 0과 1 사이의 값으로 압축합니다.

$$f(Z) = \frac{1}{1 + e^{-Z}}$$

- 특징: 출력이 확률로 해석될 수 있어 이진 분류(binary classification) 문제의 출력층에 주로 사용됩니다.
- 문제점: 입력값이 매우 크거나 작으면 기울기가 0에 가까워져 기울기 소실 문제가 발생할 수 있습니다.

6. 하이퍼볼릭 탄젠트(Tanh)

- 정의: 입력 Z 를 -1과 1 사이의 값으로 압축합니다.

$$f(Z) = \frac{e^Z - e^{-Z}}{e^Z + e^{-Z}}$$

- 특징: 시그모이드와 유사하지만, 출력이 0을 중심으로 대칭이므로 학습에 더 유리할 수 있습니다. 시그모이드와 마찬가지로 기울기 소실 문제가 있습니다.

7. 지수 선형 단위(ELU: Exponential Linear Unit)

- 정의: 음수 영역에서 지수 함수 형태를 가집니다.

$$f(Z) = \begin{cases} \alpha(e^Z - 1) & \text{if } Z < 0 \\ Z & \text{if } Z \geq 0 \end{cases}$$

- 특징: ReLU의 장점을 가지면서도 음수 영역에서 부드러운 변화를 보여 '죽은 ReLU' 문제를 완화합니다.

□ 핵심 정보

활성화 함수 선택의 중요성: 이론적으로는 비선형 활성화 함수를 충분히 많이 사용하면 어떤 함수라도 근사할 수 있으므로, 어떤 활성화 함수를 선택하든 큰 차이가 없을 수 있습니다. 하지만 실제 구현 및 최적화 관점에서는 활성화 함수의 선택이 학습 속도, 수렴 여부, 지역 최저점(local minimum)에 빠지는 경향 등 성능에 큰 영향을 미칠 수 있습니다. 따라서 실제 문제에서는 다양한 활성화 함수를 실험해보고 최적의 것을 찾는 것이 중요합니다.

신경망 유형: 회귀와 분류

신경망은 출력층의 활성화 함수와 뉴런 수에 따라 다양한 문제를 해결할 수 있습니다.

1. 회귀 (Regression)

- 목표: 연속적인 숫자 값을 예측합니다 (예: 주택 가격, 온도).
- 출력층:
 - 단일 스칼라 출력: 출력 뉴런 1개, 활성화 함수 없음(선형 출력) 또는 ReLU (0 이상의 값 예측).
 - 다차원 벡터 출력: 출력 뉴런 M 개, 활성화 함수 없음(선형 출력).
- Keras 예시:

```
1 from tensorflow import keras
2 from tensorflow.keras import layers
3
4 model = keras.Sequential([
5     layers.Dense(16, activation='relu', input_shape=(n,)), # 은닉층: 개 16 뉴
      런, ReLU 활성화
6     layers.Dense(1) # 출력층: 개 1 뉴런스칼라 ( 출력), 활성화함수없음선형 ( )
7 ])
```

Listing 1: Keras 회귀 모델 예시 (ReLU 활성화 함수 사용)

이 모델은 0 이상의 값을 출력할 수 있습니다. 만약 음수 값도 예측해야 한다면 출력층에 활성화 함수를 지정하지 않거나 linear를 사용합니다.

2. 분류 (Classification)

- 목표: 입력 데이터가 어떤 범주(클래스)에 속하는지 예측합니다 (예: 고양이/개, 긍정/부정).
- 출력층:
 - 이진 분류 (2개 클래스): 출력 뉴런 1개, 시그모이드(sigmoid) 활성화 함수. 출력은 0과 1 사이의 확률입니다.
 - 다중 분류 (M 개 클래스): 출력 뉴런 M 개, 소프트맥스(softmax) 활성화 함수. 출력은 각 클래스에 속할 확률 분포(총합이 1)입니다.
- Keras 예시:

```
1 from tensorflow import keras
2 from tensorflow.keras import layers
3
4 model = keras.Sequential([
```

```
5 layers.Dense(16, activation='relu', input_shape=(n,)), # 은닉층: 개 16 뉴  
   런, ReLU 활성화  
6 layers.Dense(M, activation='softmax') # 출력층: 개 M 뉴런 (개 M 클래  
   스), Softmax 활성화  
7 ])
```

Listing 2: Keras 분류 모델 예시 (Softmax 활성화 함수 사용)

손실 함수 (Loss Functions)

개념 및 역할

손실 함수(Loss Function)는 모델의 예측($\hat{y}$)이 실제 관측 값(y)과 얼마나 다른지 측정하는 함수입니다.

- **개별 손실 (L_i):** 하나의 데이터 포인트(x_i, y_i)에 대한 예측($\hat{y}_i$)과 실제 값(y_i) 사이의 차이를 측정합니다.
- **비용 함수 (J):** 전체 데이터셋에 대한 개별 손실들의 평균입니다. 신경망 학습의 목표는 이 비용 함수를 최소화하는 것입니다.

$$J(W) = \frac{1}{M} \sum_{i=1}^M L_i(W, x_i, y_i)$$

여기서 M 은 데이터 포인트의 총 개수, W 는 신경망의 모든 파라미터(가중치와 편향)를 나타냅니다.

손실 함수의 선택은 해결하려는 문제의 유형(회귀 또는 분류)에 따라 달라집니다.

회귀 문제용 손실 함수

회귀 문제는 연속적인 숫자 값을 예측하는 문제이므로, 예측 값과 실제 값 사이의 '거리'를 측정하는 손실 함수를 사용합니다.

1. 제곱 오차 (Squared Error)

- **정의:** 예측 값과 실제 값의 차이를 제곱한 것입니다.
 - 1차원 출력: $L_i = (\hat{y}_i - y_i)^2$
 - 다차원 출력 (M 차원): $L_i = \sum_{j=1}^M (\hat{y}_{ij} - y_{ij})^2$ (유클리드 거리의 제곱)
- **비용 함수:** 평균 제곱 오차(Mean Squared Error, MSE)라고 불립니다.

$$\text{MSE} = \frac{1}{M} \sum_{i=1}^M (\hat{y}_i - y_i)^2$$

- **특징:** 오차가 클수록 더 큰 페널티를 부여합니다. 미분 가능하여 최적화에 용이합니다.

2. 절대 오차 (Absolute Error)

- **정의:** 예측 값과 실제 값의 차이에 절댓값을 취한 것입니다.
 - 1차원 출력: $L_i = |\hat{y}_i - y_i|$
 - 다차원 출력 (M 차원): $L_i = \sqrt{\sum_{j=1}^M (\hat{y}_{ij} - y_{ij})^2}$ (유클리드 거리)
- **비용 함수:** 평균 절대 오차(Mean Absolute Error, MAE)라고 불립니다.

$$\text{MAE} = \frac{1}{M} \sum_{i=1}^M |\hat{y}_i - y_i|$$

- **특징:** 이상치(outlier)에 덜 민감합니다. 0에서 미분 불가능하지만, 실제 최적화에서는 큰 문제가 되지 않습니다.

두 함수의 비교

- **최저점 위치:** MSE와 MAE 모두 최적의 파라미터 W 를 찾으면 동일한 전역 최저점(global minimum)을 가리킵니다. 즉, 한 함수를 최소화하는 W 는 다른 함수도 최소화합니다.
- **수렴 특성:** 두 함수의 '모양(shape)'이 다르기 때문에, 최적화 알고리즘의 수렴 속도와 지역 최저점에 빠지는 경향이 달라집니다.
 - **MSE:** 오차가 클 때 기울기가 커서 빠르게 움직이고, 최저점에 가까워질수록 기울기가 작아져 정밀하게 수렴합니다.
 - **MAE:** 기울기가 일정하여 최저점에 가까워져도 스텝 크기가 크게 줄어들지 않아 최저점 주변에서 진동할 수 있습니다.
- **실제 적용:** 실제로는 전역 최저점을 찾기 어렵고 지역 최저점에 빠질 가능성이 높기 때문에, 두 손실 함수 중 어떤 것이 더 나은 수렴 경로를 제공하는지 실험해보고 선택해야 합니다.

분류 문제용 손실 함수

분류 문제는 데이터가 특정 범주에 속할 '확률'을 예측하는 문제이므로, 확률 분포 간의 차이를 측정하는 손실 함수를 사용합니다.

1. 원-핫 인코딩 (One-Hot Encoding)

- **개념:** 범주형 데이터를 컴퓨터가 이해할 수 있는 이진 벡터 형태로 변환하는 방법입니다.
- **예시:** '고양이', '개' 두 클래스가 있다면,
 - 고양이: $[1, 0]$
 - 개: $[0, 1]$
 세 클래스('고양이', '개', '소')라면:
 - 고양이: $[1, 0, 0]$
 - 개: $[0, 1, 0]$
 - 소: $[0, 0, 1]$
- **역할:** 실제 관측 값(y)을 이러한 원-핫 벡터로 표현하여 모델의 예측($\hat{y}$, 확률 벡터)과 비교합니다.

2. 소프트맥스 (Softmax) 함수

- **개념:** 다중 분류 문제의 출력층에 사용되는 활성화 함수로, 신경망의 최종 출력을 각 클래스에 속할 확률 분포로 변환합니다.
- **정의:** M 개의 클래스가 있을 때, 각 클래스 k 에 대한 확률 $\hat{y}_k$ 는 다음과 같이 계산됩니다.

$$\hat{y}_k = \frac{e^{Z_k}}{\sum_{j=1}^M e^{Z_j}}$$

여기서 Z_k 는 신경망의 이전 층에서 나온 선형 결합 값(로짓)입니다.

- **특징:**
 - 모든 $\hat{y}_k$ 는 0보다 크고 1보다 작습니다.
 - 모든 $\hat{y}_k$ 의 합은 1입니다 ($\sum_{k=1}^M \hat{y}_k = 1$).
- **역할:** 신경망이 '고양이일 확률 0.9, 개일 확률 0.1'과 같은 형태로 예측을 출력할 수 있게 합니다.

3. 분류 문제에서 MSE가 부적합한 이유

분류 문제에서 MSE를 손실 함수로 사용하는 것은 일반적으로 좋지 않은 결과를 초래합니다.

- **이질적인 값 비교:** MSE는 연속적인 확률 값(예측 $\hat{y}$)과 이산적인 이진 값(원-핫 인코딩 y) 사이의 차이를 측정합니다. 이는 본질적으로 다른 유형의 값을 비교하는 것입니다.
- **비선형성 증가:** 이러한 이질적인 값의 차이를 제공하면, 파라미터 W 에 대한 비용 함수가 매우 비선형적이고 복잡한 형태를 띠게 됩니다.
- **최적화 어려움:** 결과적으로 MSE를 사용하여 분류 문제를 최적화하려고 하면, 알고리즘이 쉽게 지역 최저점에 빠져 전역 최저점을 찾기 매우 어려워집니다.

4. 교차 엔트로피 (Cross-Entropy Loss)

분류 문제에서 가장 널리 사용되는 손실 함수입니다.

- **정의:** 예측 확률 분포($\hat{y}$)가 실제 레이블 분포(y)와 얼마나 다른지 측정합니다.
 - **이진 교차 엔트로피 (Binary Cross-Entropy):** 2개 클래스 분류에 사용됩니다.

$$L_i = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

여기서 y_i 는 0 또는 1, $\hat{y}_i$ 는 0과 1 사이의 확률입니다.

- **범주형 교차 엔트로피 (Categorical Cross-Entropy):** M 개 이상의 클래스 분류에 사용됩니다.

$$L_i = - \sum_{k=1}^M y_{ik} \log(\hat{y}_{ik})$$

여기서 y_{ik} 는 i 번째 데이터의 실제 레이블(원-핫 벡터), $\hat{y}_{ik}$ 는 i 번째 데이터의 k 번째 클래스 예측 확률입니다.

- **예시 (범주형 교차 엔트로피):**
 - 실제 레이블 $y = [1, 0]$ (고양이)
 - 예측 $\hat{y} = [0.99, 0.01]$ (고양이일 확률 99%)

$$L = -[1 \cdot \log(0.99) + 0 \cdot \log(0.01)] \approx -[1 \cdot (-0.01)] = 0.01$$

손실이 매우 작습니다.

- 실제 레이블 $y = [1, 0]$ (고양이)
- 예측 $\hat{y} = [0.01, 0.99]$ (개일 확률 99%)

$$L = -[1 \cdot \log(0.01) + 0 \cdot \log(0.99)] \approx -[1 \cdot (-4.6)] = 4.6$$

손실이 매우 큼니다.

- **장점:**
 - **강력한 페널티:** 모델이 잘못된 클래스를 높은 확신으로 예측할 때 매우 큰 손실을 부여합니다.
 - **최적화 용이성:** MSE보다 파라미터에 대한 비용 함수의 형태가 더 부드럽고 볼록하여 최적화가 훨씬 용이합니다.
 - **수치적 안정성:** 소프트맥스 함수는 0을 출력하지 않으므로, $\log(0)$ 으로 인한 무한대 값은 실제로는

발생하지 않습니다.

□ 핵심 정보

손실 함수와 메트릭(Metric)의 차이:

- **손실 (Loss):** 신경망이 '최소화'해야 할 대상입니다. 학습 과정에서 파라미터를 업데이트하는 데 사용됩니다.
- **메트릭 (Metric):** 모델의 '성능을 평가'하는 데 사용됩니다. 학습 과정 중 주기적으로 계산되어 사용자에게 모델의 상태를 보여주지만, 직접적으로 최소화되는 대상은 아닙니다 (예: 정확도 (accuracy)).

최적화 알고리즘 (Optimizers)

개념 및 역할

최적화 알고리즘(Optimizer)은 신경망의 파라미터(가중치 W)를 조정하여 비용 함수($J(W)$)를 최소화하는 방법을 정의합니다. 신경망의 비용 함수는 일반적으로 매우 복잡하고 비볼록(non-convex)한 형태를 가지며, 수많은 지역 최저점(local minimum)을 포함합니다. 따라서 최적화 알고리즘은 전역 최저점(global minimum)에 최대한 가깝게 도달하는 효율적인 방법을 찾아야 합니다.

Figure 2: 비용 함수의 복잡한 지형 (예시)

(이미지 설명: 파라미터 공간(W)에서 비용 함수(J)가 여러 개의 봉우리와 골짜기(지역 최저점)를 가진 복잡한 3D 지형처럼 그려진 그림. 전역 최저점을 찾는 것이 목표임을 시각적으로 보여줌.)

경사 하강법 (Gradient Descent, GD)

원리

경사 하강법은 비용 함수의 기울기(gradient)를 계산하여, 기울기가 가리키는 반대 방향(가장 가파르게 내려가는 방향)으로 파라미터를 업데이트하는 방법입니다.

- **기울기 (Gradient):** 각 파라미터에 대한 비용 함수의 편미분 값들을 모아놓은 벡터입니다. 기울기는 함수가 가장 빠르게 증가하는 방향을 가리킵니다.

$$\nabla J(W) = \left[\frac{\partial J}{\partial W_1}, \frac{\partial J}{\partial W_2}, \dots, \frac{\partial J}{\partial W_D} \right]$$

- **파라미터 업데이트:**

$$W_{\text{새로운}} = W_{\text{이전}} - \alpha \cdot \nabla J(W_{\text{이전}})$$

여기서 α 는 학습률(learning rate)로, 한 번에 얼마나 크게 이동할지 결정하는 하이퍼파라미터입니다.

문제점

- **높은 계산 비용:** 한 번의 파라미터 업데이트를 위해 전체 데이터셋(M 개의 모든 관측치)에 대한 손실 함수의 기울기를 계산해야 합니다. 데이터셋이 클수록 한 스텝에 드는 계산 비용이 매우 커집니다.
- **지역 최저점:** GD는 결정론적(deterministic) 알고리즘이므로, 시작점에 따라 가장 가까운 지역 최저점에 갇힐 수 있습니다.
- **학습률의 중요성:** α 가 너무 크면 최저점을 지나쳐 발산할 수 있고, 너무 작으면 수렴하는 데 매우 오랜 시간이 걸립니다.

확률적 경사 하강법 (Stochastic Gradient Descent, SGD)

원리

SGD는 GD의 높은 계산 비용 문제를 해결하기 위해 고안되었습니다. 한 번의 파라미터 업데이트를 위해 전체 데이터셋이 아닌, '단 하나의 무작위로 선택된 데이터 포인트'에 대한 손실 함수의 기울기만

계산하여 사용합니다.

- **파라미터 업데이트:**

$$W_{\text{새로운}} = W_{\text{이전}} - \alpha \cdot \nabla L_i(W_{\text{이전}})$$

여기서 L_i 는 i 번째 데이터 포인트에 대한 손실 함수입니다.

장점

- **계산 효율성:** 각 업데이트 스텝의 계산 비용이 매우 저렴합니다.
- **지역 최저점 탈출:** 단일 데이터 포인트의 기울기를 사용하므로 업데이트 방향에 무작위성(stochasticity)이 생깁니다. 이 무작위성은 알고리즘이 얇은 지역 최저점에서 벗어나는 데 도움을 줍니다.

문제점

- **높은 변동성:** 업데이트 방향이 매우 불규칙하고 노이즈가 심하여, 수렴 경로가 불안정하고 최저점 주변에서 크게 진동할 수 있습니다. 이는 수렴하는 데 오랜 시간이 걸리거나 최저점에 정확히 도달하기 어렵게 만듭니다.
- **병렬 처리의 어려움:** 각 스텝마다 파라미터가 업데이트되므로, 다음 스텝의 기울기 계산은 이전 스텝의 업데이트된 파라미터에 의존합니다. 이로 인해 병렬 계산이 어렵습니다.

미니 배치 경사 하강법 (Mini-Batch Gradient Descent)

미니 배치 경사 하강법은 GD와 SGD의 장점을 결합한 가장 널리 사용되는 최적화 알고리즘입니다.

원리

한 번의 파라미터 업데이트를 위해 전체 데이터셋이나 단일 데이터 포인트가 아닌, '작은 묶음(미니 배치)'의 데이터 포인트에 대한 손실 함수의 평균 기울기를 계산하여 사용합니다.

- **파라미터 업데이트:**

$$W_{\text{새로운}} = W_{\text{이전}} - \alpha \cdot \frac{1}{B} \sum_{j \in \text{batch}} \nabla L_j(W_{\text{이전}})$$

여기서 B 는 미니 배치 크기(mini-batch size)입니다.

장점

- **계산 효율성:** SGD보다는 스텝당 계산 비용이 높지만, GD보다는 훨씬 효율적입니다.
- **변동성 제어:** SGD보다 업데이트 방향의 노이즈가 적어 수렴이 더 안정적입니다.
- **병렬 처리:** 미니 배치 내의 모든 데이터 포인트에 대한 기울기 계산은 동일한 파라미터 W 를 기준으로 이루어지므로 병렬 처리가 가능합니다. 이는 GPU와 같은 하드웨어 가속을 효과적으로 활용할 수 있게 합니다.
- **지역 최저점 탈출:** 여전히 어느 정도의 무작위성을 유지하여 얇은 지역 최저점에서 벗어나는 데 도움을 줍니다.

미니 배치 크기의 중요성

미니 배치 크기는 최적화 과정의 핵심 하이퍼파라미터입니다.

- **작은 배치 크기 (예: 32, 64):**
 - 더 많은 무작위성: SGD에 가까워져 지역 최저점 탈출에 유리합니다.
 - 스텝당 계산은 빠르지만, 전체 데이터셋을 한 번 처리하는 데 더 많은 스텝이 필요합니다.
- **큰 배치 크기 (예: 1024 이상):**
 - 더 적은 무작위성: GD에 가까워져 수렴 경로가 안정적이지만, 지역 최저점에 갇힐 가능성이 높습니다.
 - 스텝당 계산은 느리지만, 전체 데이터셋을 한 번 처리하는 데 필요한 스텝 수가 적습니다.
- **수프 비유:** 수프 맛을 보기 위해 스푼으로 떠먹는 것에 비유할 수 있습니다. 냄비 크기(데이터셋 크기)와 상관없이, 스푼 크기(미니 배치 크기)가 적절해야 정확하게 맛을 볼 수 있습니다. 즉, 최적화의 무작위성은 데이터셋 크기가 아닌 미니 배치 크기에 의해 주로 결정됩니다.
- **일반적인 선택:** 32, 64, 128과 같은 2의 거듭제곱이 일반적으로 사용됩니다. 이는 GPU 메모리 구조와 관련이 있습니다.

다른 최적화 기법

미니 배치 경사 하강법을 기반으로 다양한 고급 최적화 기법들이 개발되었습니다. 이들은 학습률을 동적으로 조절하거나, 과거 기울기 정보를 활용하여 수렴 속도를 높이고 안정성을 개선합니다.

- **모멘텀 (Momentum):** 과거 업데이트 방향에 관성(momentum)을 부여하여 진동을 줄이고 수렴을 가속화합니다.
- **네스테로프 가속 경사 (Nesterov Accelerated Gradient, NAG):** 모멘텀과 유사하지만, 미리 한 발짝 내딛은 후의 기울기를 계산하여 더 정확한 업데이트를 수행합니다.
- **적응형 학습률 기법 (Adaptive Learning Rate Methods):**
 - **Adagrad, RMSprop, Adadelta:** 각 파라미터마다 고유한 학습률을 적용하여, 자주 업데이트되는 파라미터는 학습률을 줄이고, 드물게 업데이트되는 파라미터는 학습률을 늘립니다.
 - **Adam (Adaptive Moment Estimation):** RMSprop과 Momentum의 장점을 결합한 가장 인기 있고 강력한 최적화 알고리즘 중 하나입니다. 대부분의 경우 좋은 기본 성능을 제공합니다.

□ 핵심 정보

최적화 알고리즘 선택:

- **기본 선택:** Adam 옵티마이저는 대부분의 문제에서 좋은 성능을 보여주므로, 기본적으로 사용하기 좋습니다.
- **실험:** SGD, Adam 등 다양한 옵티마이저와 그들의 하이퍼파라미터(학습률, 배치 크기 등)를 변경하며 실험하여 문제에 가장 적합한 것을 찾아야 합니다.
- **문서 참고:** 각 옵티마이저의 파라미터에 대한 자세한 정보는 Keras와 같은 라이브러리 문서를 참고하세요.

실습/코드/오류와 해결

과제 1 예시

첫 번째 과제는 신경망의 기본 개념을 이해하고 파이썬으로 구현하는 능력을 평가합니다.

□ 예제

퀴즈 예시:

- 주어진 신경망 구조에서 순전파(forward propagation)를 통해 출력을 계산합니다.
- 주어진 파이썬 구현 코드에서 신경망의 파라미터(가중치 및 편향) 수를 계산합니다.

□ 예제

과제 예시:

- 특정 신경망을 사용하여 검증 정확도(validation accuracy)를 계산하고 보고합니다.
- 다양한 최적화 알고리즘(optimizers)과 하이퍼파라미터(hyperparameters)를 실험하여 모델 성능을 비교합니다.

과제 제출 형식

과제는 다음 두 가지 형태로 제출해야 합니다.

- **보고서 (PDF/Word):** 문제 해결 과정, 주요 코드 스냅샷, 결과(플롯, 표), 그리고 결과에 대한 간략한 논의를 포함합니다. 주피터 노트북을 PDF로 변환하여 제출하는 것도 허용됩니다.
- **원본 코드 (Jupyter Notebook 또는 ZIP 파일):** TA가 코드를 직접 실행하여 검토할 수 있도록 합니다. 단일 주피터 노트북 파일인 경우 그대로 제출하고, 여러 개의 파이썬 파일인 경우 하나의 ZIP 파일로 압축하여 제출합니다.

□ 핵심 정보

보고서 작성 팁: 과제를 수행하면서 중요한 코드 부분, 중간 결과, 최종 플롯 등을 스냅샷으로 찍어 보고서에 삽입하고, 각 결과에 대한 간략한 설명을 덧붙이세요. 이는 나중에 자신의 작업을 다시 검토할 때 매우 유용합니다.

과제 해결 가이드

- Canvas에 게시된 예제 파일을 다운로드하여 실행해봅니다.
- 예제 코드를 기반으로 문제에서 요구하는 수정 사항을 적용합니다.
- 다양한 최적화 알고리즘(예: SGD, Adam)과 하이퍼파라미터(예: 학습률, 미니 배치 크기)를 실험하여 성능 변화를 관찰합니다.

□ 예제

최적화 알고리즘 변경 예시 (Keras):

```
1 from tensorflow.keras import optimizers
```

```
2
3 # SGD 옵티마이저사용학습률 ( 0.01, 미니배치크기 32)
4 # model.compile(optimizer=optimizers.SGD(learning_rate=0.01),
5 #               loss='categorical_crossentropy',
6 #               metrics=['accuracy'])
7
8 # Adam 옵티마이저사용기본 ( 학습률)
9 model.compile(optimizer='adam', # 또는 optimizers.Adam(learning_rate=0.001)
10               loss='categorical_crossentropy',
11               metrics=['accuracy'])
```

Listing 3: Keras 옵티마이저 설정 예시

optimizer 매개변수에 'sgd' 또는 'adam' 과 같은 문자열을 직접 전달하거나, `optimizers.SGD()` 또는 `optimizers.Adam()` 객체를 생성하여 `learning_rate` .

체크리스트

학습 점검표

- 강의 운영 방식 및 소통 채널을 이해했는가?
- 과제, 퀴즈, 최종 프로젝트의 마감일과 평가 기준을 숙지했는가?
- AI 도구 사용 정책을 이해하고 인용 의무를 인지했는가?
- 신경망의 기본 아이디어(자동 특징 학습, 비선형성)를 이해했는가?
- 활성화 함수의 역할과 주요 종류(ReLU, Leaky ReLU, Softmax)를 설명할 수 있는가?
- 손실 함수의 개념과 회귀/분류 문제에 적합한 손실 함수(MSE, MAE, Cross-Entropy)를 구분할 수 있는가?
- 최적화 알고리즘(GD, SGD, Mini-Batch GD)의 원리와 장단점을 이해했는가?
- 미니 배치 크기가 최적화에 미치는 영향을 설명할 수 있는가?

과제 제출 점검표

- 과제 마감일을 확인하고 보스턴 시간 기준으로 제출했는가?
- PDF 보고서에 모든 요구사항(코드 스냅샷, 결과, 플롯, 설명)을 포함했는가?
- 원본 코드(Jupyter Notebook 또는 ZIP 파일)를 함께 제출했는가?
- AI 도구를 사용했다면 보고서에 명확히 인용했는가?
- 특정 성능 목표가 있다면 달성했는지 확인했는가?

시험 대비 로드맵

- **기초 개념 숙지:** 신경망, 활성화 함수, 손실 함수, 최적화 알고리즘의 정의와 '왜 필요한지'를 이해합니다.
- **원리 이해:** 각 개념의 작동 원리(예: 경사 하강법의 파라미터 업데이트 방식, 교차 엔트로피의 손실 계산)를 파악합니다.
- **코드 실습:** 제공된 예제 코드를 직접 실행하고 수정하며 개념을 코드로 구현하는 방법을 익힙니다.
- **하이퍼파라미터 실험:** 다양한 하이퍼파라미터(학습률, 배치 크기)를 변경하며 모델 성능에 미치는 영향을 관찰합니다.
- **오류 분석:** 과제 수행 중 발생한 오류의 원인을 파악하고 해결하는 과정을 통해 문제 해결 능력을 기릅니다.

FAQ

- Q: 퀴즈 마감 기한을 놓쳤는데, 재응시할 수 있나요? A: 아니요, 퀴즈 마감 기한은 매우 엄격하며, 마감 후에는 재응시나 대체 퀴즈가 제공되지 않습니다. 퀴즈 질문이 다음 강의에서 논의되기 때문입니다. 7일의 응시 기간 동안 시간을 내어 제출해야 합니다.
- Q: 최종 프로젝트 마감 기한도 퀴즈처럼 엄격한가요? A: 네, 최종 프로젝트 마감 기한은 학교 정책상 매우 엄격합니다. 마감 기한을 놓칠 경우, 학교의 승인을 받아야만 연장이 가능합니다. 미리 계획하고 준비하는 것이 중요합니다.
- Q: 강의에서 특정 교재를 따라가야 하나요? A: 아니요, 특정 교재를 필수로 지정하지 않습니다. 강의는 '주제' 중심으로 진행되므로, 여러 책이나 온라인 자료, 연구 논문 등 다양한 출처를 통해 해당 주제를 깊이 이해하는 데 집중하는 것을 권장합니다.
- Q: 신경망에서 활성화 함수를 사용하는 이유가 무엇인가요? A: 활성화 함수는 신경망에 '비선형성'을 도입하여 복잡한 패턴과 관계를 학습할 수 있도록 합니다. 만약 활성화 함수가 없다면, 아무리 층을 깊게 쌓아도 결국 선형 모델과 동일한 결과를 내게 됩니다.
- Q: 분류 문제에서 MSE(평균 제곱 오차) 대신 교차 엔트로피(Cross-Entropy)를 사용하는 이유가 무엇인가요? A: MSE는 연속적인 예측 확률과 이산적인 실제 레이블(원-핫 인코딩) 사이의 차이를 측정하므로, 비용 함수가 매우 비선형적이고 최적화하기 어렵습니다. 반면 교차 엔트로피는 확률 분포 간의 차이를 효과적으로 측정하며, 잘못된 예측에 큰 페널티를 부여하여 최적화가 훨씬 용이합니다.
- Q: 미니 배치 크기는 어떻게 선택해야 하나요? A: 미니 배치 크기는 하이퍼파라미터이며, 문제와 데이터셋의 특성에 따라 최적의 값이 달라집니다. 일반적으로 32, 64, 128과 같은 2의 거듭제곱이 많이 사용됩니다. 작은 배치 크기는 무작위성을 높여 지역 최저점 탈출에 유리하고, 큰 배치 크기는 수렴 경로를 안정화하지만 지역 최저점에 갇힐 위험이 있습니다. 실험을 통해 최적의 값을 찾아야 합니다.

빠르게 훑어보기 (1페이지 요약)

CSCI89 Lecture 1 핵심 요약

강의 목표: 신경망(NN) 및 자연어 처리(NLP) 소개.

소통: Piazza (공식 Q&A), WhatsApp (학생 소통), Canvas (공지).

세션: 강의(화), 교수(금), TA(주 2회). 모두 녹화 제공.

평가: 과제(65%), 퀴즈(20%), 최종 프로젝트(15%).

마감: 퀴즈(엄격), 최종 프로젝트(매우 엄격).

AI 사용: 허용, 인용 필수, 책임은 본인.

NN 동기: 수동 특징 설계 한계 극복, 자동 특징 학습.

NN 구조: 입력-은닉-출력층, 가중치, 편향, 활성화 함수.

활성화 함수: 비선형성 도입. ReLU, Leaky ReLU, Sigmoid, Softmax 등.

손실 함수: 예측 오차 측정.

– **회귀:** MSE (제곱 오차), MAE (절대 오차).

– **분류:** Cross-Entropy (교차 엔트로피). Softmax와 함께 사용. MSE는 분류에 부적합.

최적화: 비용 함수 최소화.

– **GD:** 전체 데이터 사용, 느림, 지역 최저점.

– **SGD:** 단일 데이터 사용, 빠름, 노이즈, 지역 최저점 탈출.

– **Mini-Batch GD:** 작은 배치 사용, GD와 SGD 장점 결합, 가장 일반적.

– **Adam:** 인기 있는 고급 옵티마이저.

하이퍼파라미터: 학습률(α), 미니 배치 크기. 실험을 통해 최적화.